

Model

- ◆ Overview of model
- ◆ Create model
- ◆ Passing model from controllers to views
- ◆ Explain how to create strongly typed models
- ◆ Model binding
- ◆ Scaffolding in Visual Studio.NET

Overview of Model

- ◆ In an MVC application, a model:
 - ◆ Is a class containing properties that represents data of an application.
 - ◆ Represents data associated with the application.
- ◆ The MVC pattern defines the following three types of models, where each model has specific purpose:
 - ◆ Data model: Represent classes that interact with a database. Data models are set of classes that can either follow the database-first approach or code-first approach.
 - ◆ Business model: Represent classes that implement a functionality that represents business logic of an application.
 - ◆ View model: Represent classes that provide information passed between controllers and views..

Create Model

- ◆ To create a model in an ASP.NET MVC application, you need to:
 - ◆ Create a public class.
 - ◆ Declare public properties for each information that the model represents.
- ◆ Following code snippet shows declaring a model class named User:

```
public class User
{
    public long Id { get; set; }
    public string name { get; set; }
    public string address { get; set; }
    public string email { get; set; }
}
```

Accessing a Model within a Controller

- ◆ In an ASP.NET Core MVC application when a user request for some information, the request is received by an action method.
- ◆ The action method is used to access the model storing the data.
- ◆ To access the model, you need to create an object of the model class and either retrieve or set the property values of the object.
- ◆ Following code shows the creating an object of the model class

```
public ActionResult Index()
{
    var user = new MVCModelDemo.Models.User();
    user.name = "John Smith";
    user.address = "Park Street";
    user.email = "john@mvcexample.com";
    return View();
}
```

Passing Model Data from Controller to View

- ◆ Once you have accessed the model within a controller, you need to make the model data accessible to a view so that the view can display the data to the user.
- ◆ To do this, you need to pass the model object to the view while invoking the view.
- ◆ You can model the object as follows:
 - ◆ A single object
 - ◆ A collection of model objects

Passing Model Data from Controller to View

- ◆ In an action method, you can create a model object and then pass the object to a view by using the ViewBagobject.
- ◆ Following code shows passing the User model data from an action method to a view by using a ViewBag object:

```
public ActionResult Index()
{
    var user = new MVCModelDemo.Models.User();
    user.name = "John Smith";
    user.address = "Park Street";
    user.email = "john@mvcexample.com";
    ViewBag.user = user;
    return View();
}
```

- ◆ In this code, an object of the User model class is created and initialized with values. The object is then, passed to the view by using a ViewBagobject

Passing Model Data from Controller to View

- ◆ You can access the data of the model object stored in the ViewBagobject from within the view.
- ◆ Following code snippet shows accessing the data of the model object stored in the ViewBag object::

```
<!DOCTYPE html>
<html>
<body>
    <p> User Name: @ViewBag.user.name</p>
    <p> Address: @ViewBag.user.address</p>
    <p> Email: @ViewBag.user.email</p>
</body>
</html>
```

- ◆ In this code, the view accesses and displays the name, address, and emailproperties of the Usermodel object stored in the ViewBag object..

Passing Model Data from Controller to View

- ◆ Following code shows passing a collection of model objects to a view:

```
public ActionResult Index(){
    var user = new List<User>();
    var user1 = new User();
    user1.name = "Mark Smith";
    user1.address = "Park Street";
    user1.email = "Mark@mvcexample.com";
    var user2 = new User();
    user2.name = "John Parker";
    user2.address = "New Park";
    user2.email = "John@mvcexample.com";
    var user3 = new User();
    user3.name = "Steave Edward ";
    user3.address = "Melbourne Street";
    user3.email = "steave@mvcexample.com";
    user.Add(user1); user.Add(user2);
    user.Add(user3);
    ViewBag.user = user; return View(); }
```

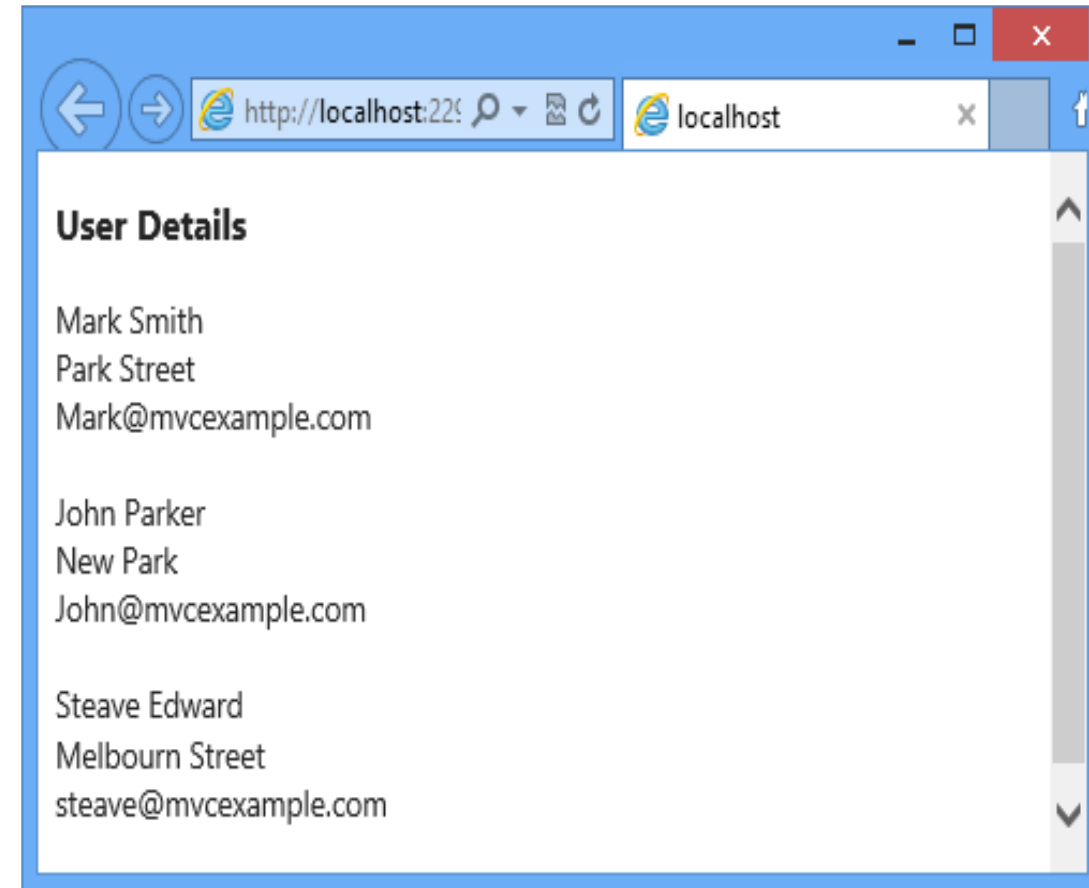
Passing Model Data from Controller to View

- ◆ Creates and initializes three objects of the model class, named User.
- ◆ Then, a List<User>collection is created and the model objects are added to it.
- ◆ Finally, the collection is passed to the view by using a ViewBag object.
- ◆ Once you pass a collection of model objects to a view using a ViewBag object:.

```
<!DOCTYPE html>
<html> <body>
<h3>User Details</h3>
@{ var user = ViewBag.user;}
@foreach (var p in user)
{
    @p.name<br />
    @p.address<br />
    @p.email<br />
    <br /> }
</body>
</html>
```

Passing Model Data from Controller to View

- ◆ Following figure shows the output of retrieving model objects
- ◆ You can use another approach to pass a collection of model objects from an action method to a view is to pass the collection directly as a parameter to the View() method.



Passing Model Data from Controller to View

- ◆ Following code snippet shows passing a collection of model objects to a view as a parameter to the View()method

```
public ActionResult Index()
{
    var user = new MVCModelDemo.Models.User();
    user.name = "John Smith";
    user.address = "Park Street";
    user.email = "john@mvcexample.com";
    return View(user);
}
```

Passing Model Data from Controller to View

◆ Type casting

```
<html> <body>
<h3>User Details</h3>
@{
var user = Model as MVCModelDemo.Models.User;
}
@user.name <br/>
@user.address<br/>
@user.email<br/>
</body> </html>
```

◆ Casting Model to MVCModelDemo.Models.User.

Using Strong Type

- ◆ You can also ignore explicit type casting of a model object, by creating a strongly typed view.

```
@model <model_name>
```

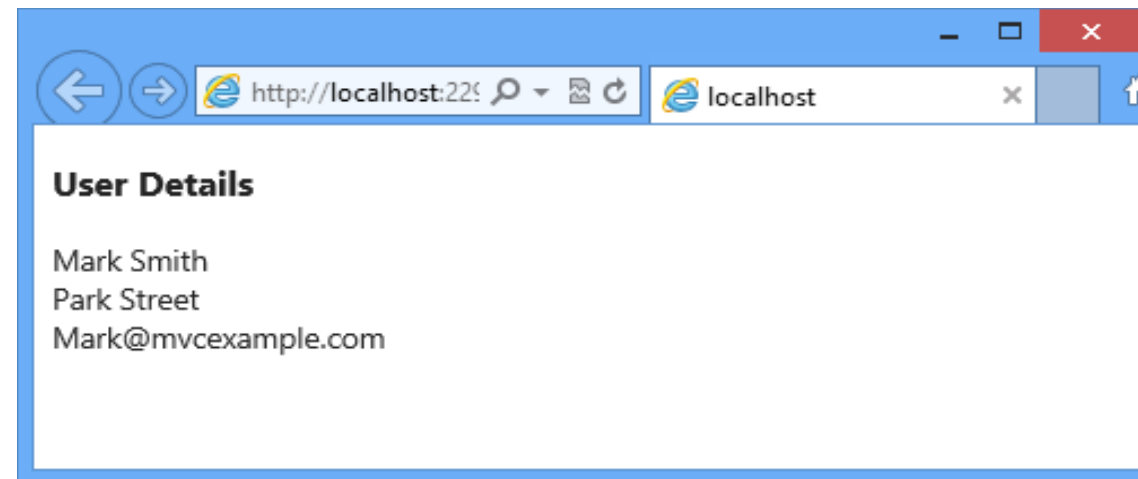
- ◆ where,
 - ◆ `model_name`: is the fully qualified name of the model class.
- ◆ Once you use the `@model` keyword, you can access the properties of the model object in the view..

Using Strong Typing

- ◆ Following code snippet shows accessing the properties of the model object by using the `@model` keyword:

```
@model MVCModelDemo.Models.User
<html><body>
<h3>User Details</h3>
    @Model.name <br/>
    @Model.address<br/>
    @Model.email<br/>
</body> </html>
```

- ◆ Following figure shows the output of accessing properties of the Modelobject:



Pass a collection model

- ◆ Return collection model to view as a parameter of View() method
- ◆ Tạo và khởi gán 3 đối tượng User.
- ◆ Create collection List<User> and add 3 User objects into collection
- ◆ Return collection to view as parameter of View() method

```
ActionResult Index() {  
    var user = new List<User>();  
    var user1 = new User();  
    user1.name = "Mark Smith";  
    user1.address = "Park Street";  
    user1.email = "Mark@mvcexample.com";  
    var user2 = new User();  
    user2.name = "John Parker";  
    user2.address = "New Park";  
    user2.email = "John@mvcexample.com";  
    var user3 = new User();  
    user3.name = "Steave Edward ";  
    user3.address = "Melbourn Street";  
    user3.email = "steave@mvcexample.com";  
    user.Add(user1);  
    user.Add(user2);  
    user.Add(user3);  
    return View(user);  
}
```

Pass a collection model

- ◆ Using @model to define strong typing:

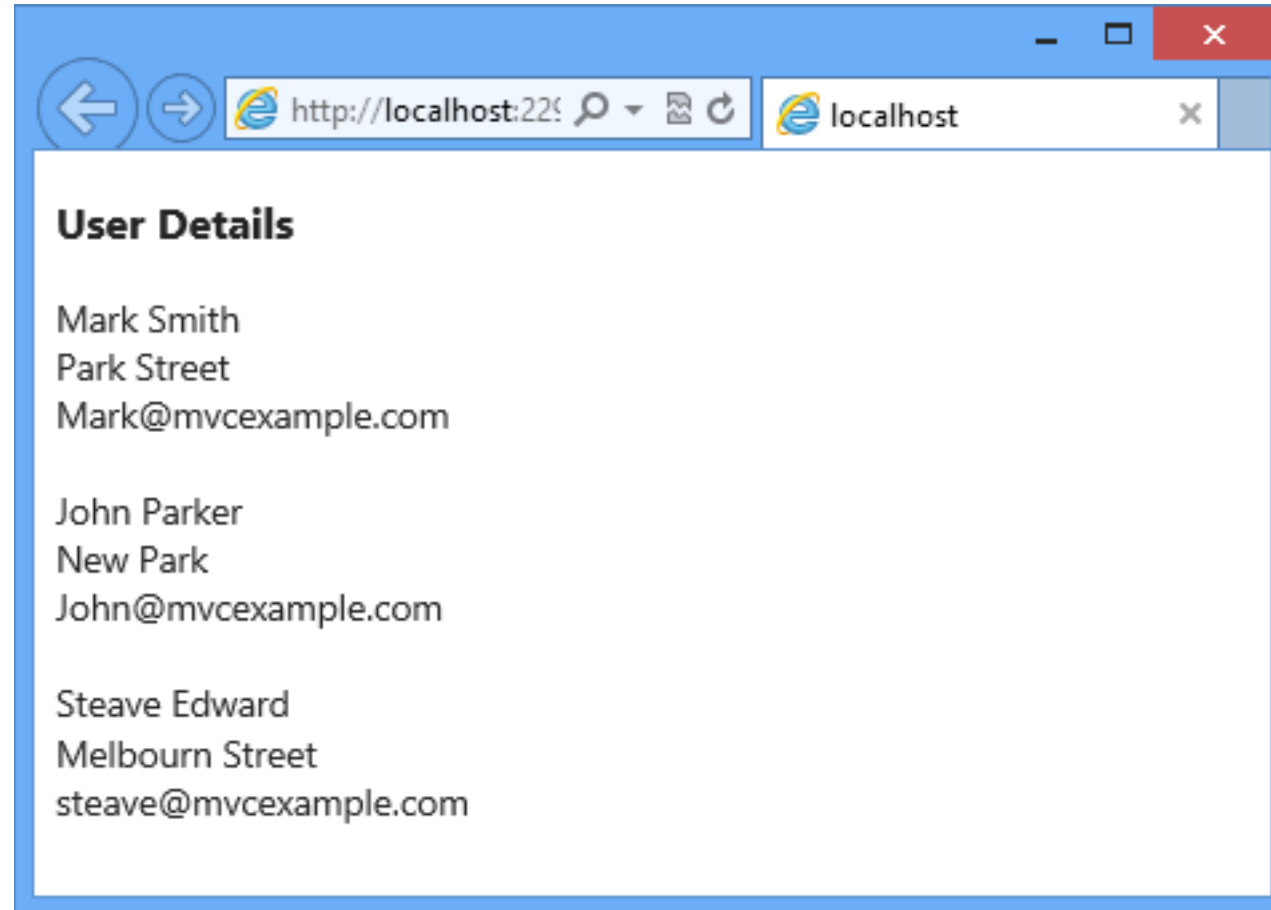
```
@model IEnumerable<MVCDemo.Models.User>
```

Pass a collection model

◆ Render collection of Users in view:

```
@model IEnumerable<MVCModelDemo.Models.User>
<html>
<body>
<h3>User Details</h3>
@{
    var user = Model;
}
@foreach (var u in user)
{
    @u.name <br/>
    @u.address<br/>
    @u.email<br/>
    <br/>
}
</body>
</html>
```

Pass a collection model



Role of Model Binder

- ◆ When a user submits information in a form within a strongly typed view, ASP.NET Core MVC automatically examines the HttpRequest object and maps the information sent to fields in the model object.
- ◆ The process of mapping the information in the HttpRequest object to the model object is known as model binding.
- ◆ Some of the advantages of model binding are as follows:
 - ◆ Automatically extract the data from the HttpRequest object.
 - ◆ Automatically converts data type.
 - ◆ Makes data validation easy..

Role of Model Binder

- ◆ The MVC Framework provides a model binder that performs model binding in application.
- ◆ The `DefaultModelBinder` class implements the model binder on the MVC Framework.
- ◆ The two most important roles of the model binder are as follows:
 - ◆ Bind request to primitive values
 - ◆ Bind request to objects

Binding Primitive values

- ◆ To understand how the model binder binds request to primitive values, consider a scenario where you are creating a log in form that accepts log in details from user.
- ◆ For this, first you need to create a Loginmodel in your application

```
public class Login
{
    public string userName { get; set; }
    [DataType(DataType.Password)]
    public string password { get; set; }
}
```

- ◆ This code creates two properties named username and password in the Login model.
- ◆ After creating the model class, you need to create an Index.cshtml view to display the login form.

Binding Primitive values

- ◆ Following code shows the content of the Index.cshtml file:

```
@model ModelDemo.Models.Login
@{ ViewBag.Title = "Index"; }
<h2>User Details</h2>
@using (Html.BeginForm()) {
    @Html.ValidationSummary(true)
    <div>@Html.LabelFor(model =>model.userName)</div>
    <div>@Html.EditorFor(model =>model.userName)</div>
    <div>@Html.LabelFor(model =>model.password)</div>
    <div>@Html.EditorFor(model =>model.password) </div>
    <div><input type="submit" value="Submit" /></div>
}
```

Binding Primitive values

- ◆ Once, you have created the view, you need to create a controller class that contains the Index() action method to display the view.

```
public class HomeController : Controller {  
    public ActionResult Index() {  
        return View();  
    }  
    [HttpPost]  
    public ActionResult Index(string userName, string password) {  
        if (userName == "Peter" && password == "pass@123") {  
            string msg = "Welcome " + userName;  
            return Content(msg);  
        }  
        else {  
            return View();  
        }  
    }  
}
```

Binding Primitive values

- ◆ In the preceding code:
 - ◆ The first Index()method returns the Index.cshtml view that displays a login form.
 - ◆ The second Index()method is marked with the HttpPost attribute. This method accepts two primitive as parameters. The Index()method compares the parameters with predefined values and returns a message if the comparison returns true. Else, the Index()method returns back the Index.cshtml view.
- ◆ When a user submits the login data, the default model binder maps the values of the username and password fields to the primitive type parameters of the Index()action method.
- ◆ In the Index() action method, you can perform the required authentication and return a result..

Binding object

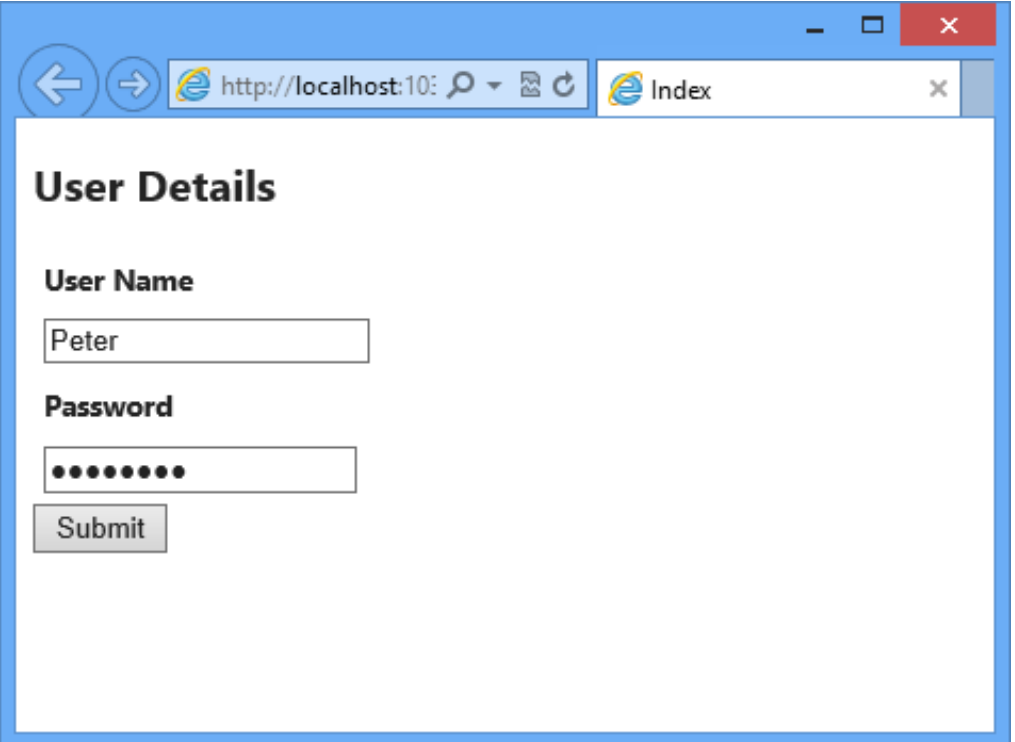
- ◆ Following code shows the updated controller class.

```
public class HomeController : Controller {  
    public ActionResult Index() {  
        return View();  
    }  
    [HttpPost]  
    public ActionResult Index(Login login) {  
        if (login.userName == "Peter" && login.password == "pass@123") {  
            string msg = "Welcome " + login.userName;  
            return Content(msg); }  
        else {  
            return View(); } }  
}
```

- ◆ The first Index() method returns the Index.cshtml view that displays a login form.
- ◆ The second Index() method automatically removes the data from the HttpRequest object and put into the Login object

Binding object

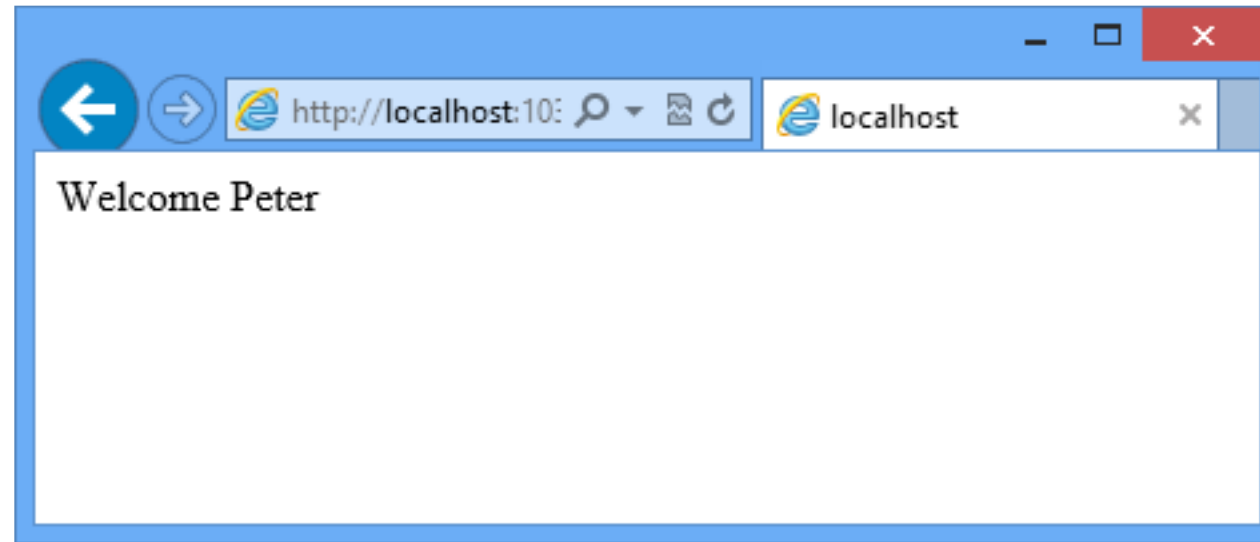
- ◆ When a user submits the login data, the Index() method validates the username and password passed in the Login object.
- ◆ When the validation is successful, the view displays a welcome message.
- ◆ When you access the application from the browser the Index.cshtml view displays the login form
 - ◆ 1. Enter Peter in text field **UserName** and pass@123 in text field **Password**.
 - ◆ 2. Click **Submit**.



The screenshot shows a web browser window with the address bar displaying 'http://localhost:10:'. The page title is 'Index'. The main content area is titled 'User Details' and contains a login form. The form has two text input fields: 'User Name' with the value 'Peter' and 'Password' with masked characters '.....'. Below the password field is a 'Submit' button.

User Details	
User Name	Peter
Password	
Submit	

Binding object



Sử dụng Scaffolding của Visual Studio.NET

- ◆ The ASP.NET MVC Framework provides a feature called scaffolding that allows you to generate views automatically.
- ◆ By convention, scaffolding uses specific name for views. After creating a view it stores the auto-generated code in respective places for the application to work.
- ◆ Following are the five types of template that provides to create views:
 - ◆ **List:** Generates markup to display the list of model objects.
 - ◆ **Create:** Generates markup to add a new object to the list.
 - ◆ **Edit:** Generates markup to edit an existing model object.
 - ◆ **Details:** Generates markup to show the information of an existing model object.
 - ◆ **Delete:** Generates markup to delete an existing model object.
- ◆ In this code, ViewData is used to display the values of the Message and CurrentTime keys.

List template

- ◆ List template allows you to create a view that displays a list of model objects.
- ◆ Following code shows an Index() action method that returns an ActionResult object through a call to the View() method of the controller class:

```
public ActionResult Index()
{
    var user = new List<User>();
    //Code to populate the user collection
    return View(user);
}
```

- ◆ This code shows the Index() action method of a controller that returns the result of a call to the View() method.
- ◆ The result of the View() method is an ActionResult object that renders a view.
- ◆ In this code, ViewData is used to display the values of the Message and CurrentTime keys.

List template

- ◆ To create a view using the List template, you need to perform the following steps:
- ◆ 1.Right-click inside the action method for which you need to create a view.
- ◆ 2.Select **Add View** from the context menu that appears. The **Add View** dialog box is displayed, as shown in the following figure:

The screenshot shows the 'Add View' dialog box with the following configuration:

- View name:** Index
- View engine:** Razor (CSHTML)
- ☐ Create a strongly-typed view
- Model class:** (empty)
- Scaffold template:** Empty
- ☒ Reference script libraries
- ☐ Create as a partial view
- ☒ Use a layout or master page
- Layout or master page:** (empty field with browse button)
- (Leave empty if it is set in a Razor _viewstart file)
- ContentPlaceHolder ID:** MainContent
- Buttons:** Add, Cancel

List template

- ◆ 3. Select the **Create a strongly-typed view** check box. This will enable the fields that allow you to specify the model class and scaffolding template.
- ◆ 4. Select the model class from the **Model class** drop-down list.
- ◆ 5. Select **List** from the **Scaffold templates** drop-down list. Following figure specifies the model class and the scaffolding template:

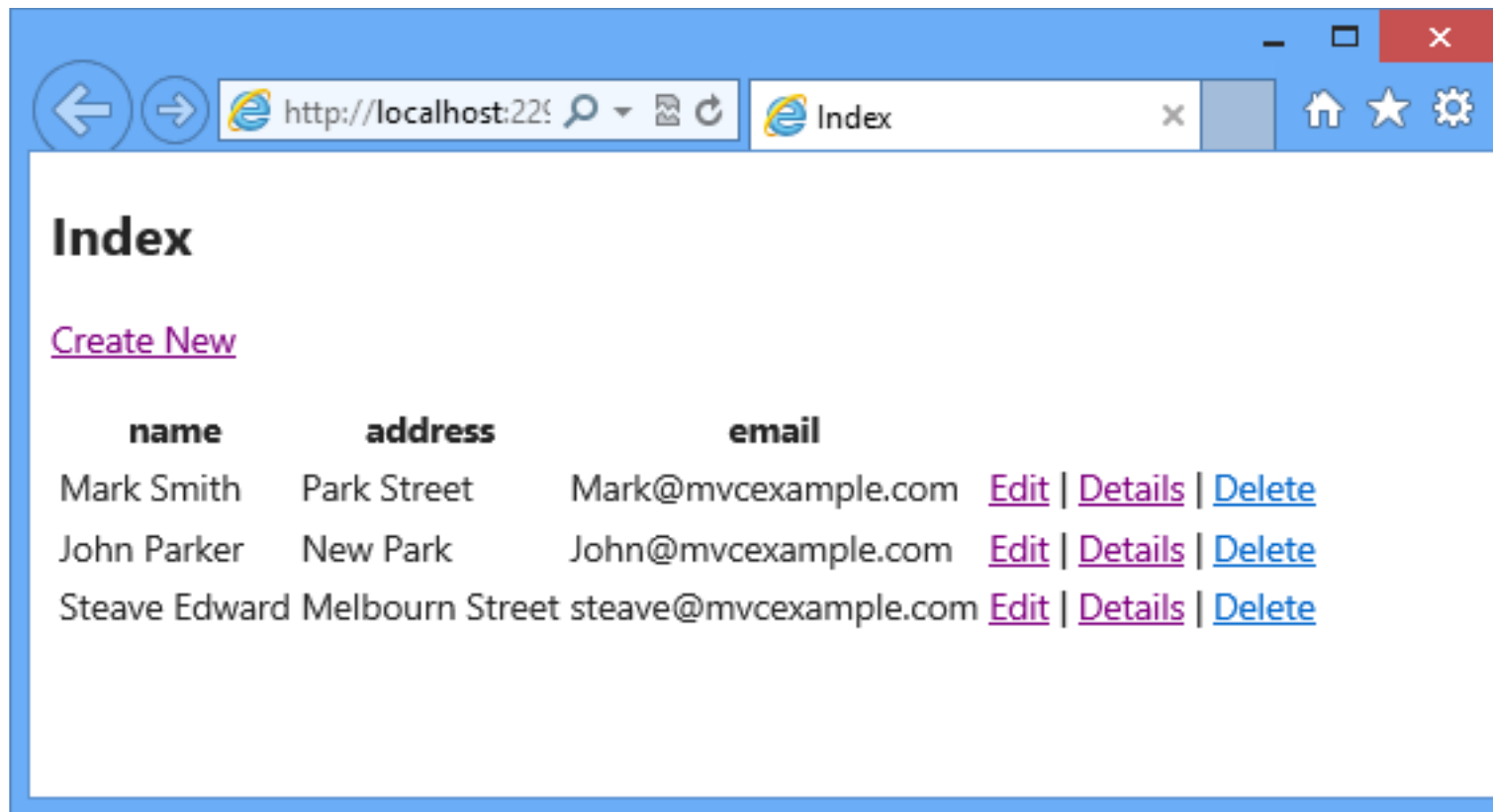
The screenshot shows the 'Add View' dialog box with the following configuration:

- View name:** Index
- View engine:** Razor (CSHTML)
- ☒ **Create a strongly-typed view**
- Model class:** User (MVCModelDemo.Models)
- Scaffold template:** List
- ☒ **Reference script libraries**
- ☐ **Create as a partial view**
- ☒ **Use a layout or master page:**
- ContentPlaceHolder ID:** MainContent

Buttons: Add, Cancel

List template

- ◆ 6. Click **Add**. Visual Studio.NET automatically creates the appropriate directory structure and adds the view file to it.
- ◆ Following figure shows the output of the auto-generated markup:



Create template

- ◆ The Create template allows you to generate a view that accepts the details of a new object to be stored in a data store.
- ◆ You need to create an action method, to display a view based on the Create template.
- ◆ Following code shows creating an action method named, Create() in the HomeController:

```
public ActionResult Create()
{
    return View();
}
```

- ◆ This code creates the Create() action method that will invoke when a user clicks the Createlink on the view generated using the List template.

Create template

- ◆ To create a view using the Create template, you need to perform the following steps:
 - ◆ 1.Right-click inside the Create() action method.
 - ◆ 2.Select **Add View** from the context menu that appears. The **Add View** dialog box appears.
 - ◆ 3.Select the **Create a strongly-typed view** checkbox.
 - ◆ 4.Select the model class from the **Model class**drop-down list.
 - ◆ 5.Select **Create from the Scaffold templates**drop-down list.
 - ◆ 6.Click **Add**. Visual Studio.NET automatically creates a view named, Create in the appropriate directory structure.

Create template

- ◆ Following code snippet shows the auto-generated markup using the Create template:

```
@model MVCModelDemo.Models.User
@{ ViewBag.Title = "Create"; }
<h2>Create</h2>
@using (Html.BeginForm()) {
    @Html.ValidationSummary(true)
    <fieldset>
        <legend>User</legend>
        <div class="editor-label">@Html.LabelFor(model => model.name)</div>
        <div class="editor-field">
            @Html.EditorFor(model => model.name)
            @Html.ValidationMessageFor(model => model.name)
        </div>
        <div class="editor-label">
            @Html.LabelFor(model => model.address)
        </div>
        <div class="editor-field">
```

Create template

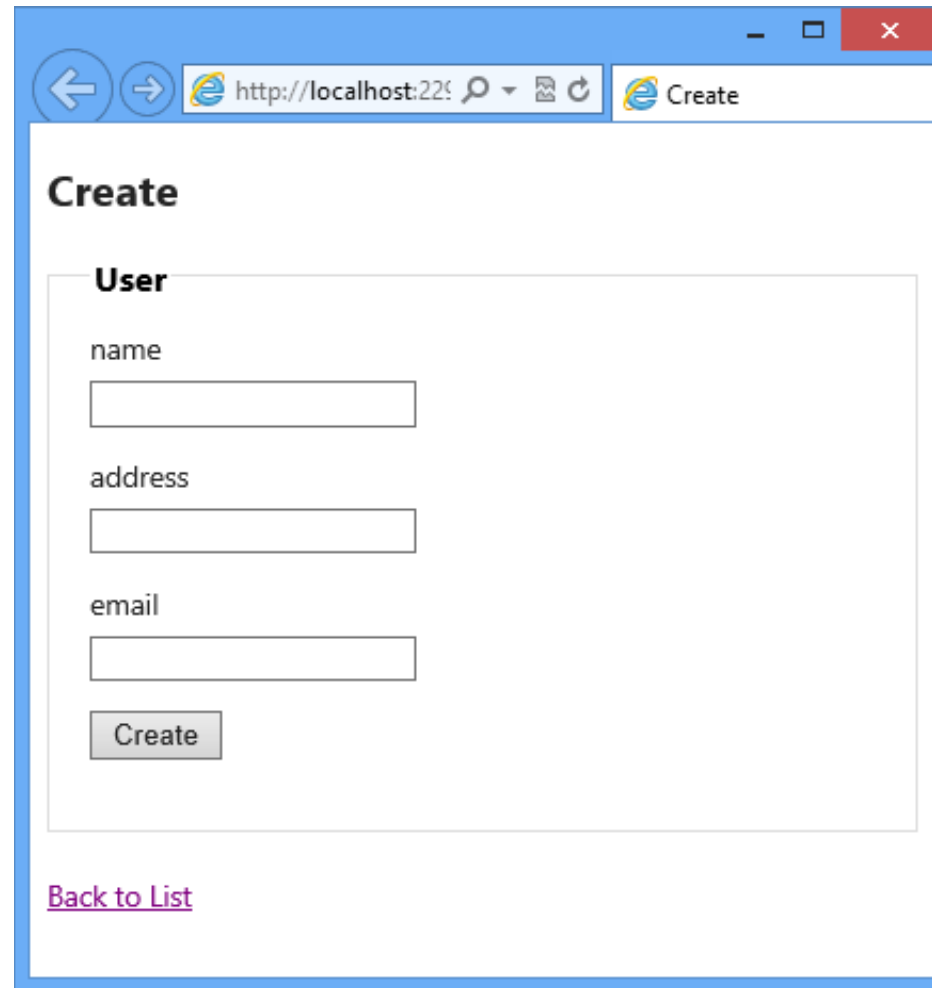
```
        @Html.EditorFor(model =>model.address)
        @Html.ValidationMessageFor(model =>model.address)
    </div>
    <div class="editor-label">@Html.LabelFor(model =>model.email)</div>
    <div class="editor-field">
        @Html.EditorFor(model =>model.email)
        @Html.ValidationMessageFor(model =>model.email)
    </div>
    <p><input type="submit" value="Create" /></p>
</fieldset>
}
<div>@Html.ActionLink("Back to List", "Index")</div>
@section Scripts {
    @Scripts.Render("~/bundles/jqueryval")
}
```

Create template

- ◆ In this code,
 - ◆ The BeginFormhelper method starts a form.
 - ◆ The Html.ValidationSummary() validation helper displays the summary of all the error messages at one place.
 - ◆ The Html.LabelFor() helper method displays an HTML label element with the name of the property.
 - ◆ The Html.EditorFor() helper method displays a textbox that accepts the value of a model property.
 - ◆ The Html.ValidationMessageFor() validation helper method displays a validation error message for the associated model property.

Create template

- ◆ Following figure shows the output of auto-generated markup using the Create template:



The screenshot shows a web browser window with the address bar displaying 'http://localhost:22...' and the page title 'Create'. The main content area is titled 'Create' and contains a form for creating a 'User'. The form has three input fields labeled 'name', 'address', and 'email', each followed by a text input box. Below these fields is a 'Create' button. At the bottom of the form, there is a link labeled 'Back to List'.

Create

User

name

address

email

[Back to List](#)

Edit Template

- ◆ The Edit template can be used whenever you want to generate a view that required to be used for modifying the details of an existing object stored in a data store.
- ◆ To display a view based on the Edit template, you need to create an action method to pass the model object to be edited to the view.
- ◆ Following code shows the action method named Edit()

```
public ActionResult Edit()  
{  
    return View();  
}
```

- ◆ Once you have created the Edit() action method in the controller, you can use Visual Studio.NET to create the view using the Edit template. To create the view using the Edit template in Visual Studio.NET, you need to select **Edit** from the **Scaffold templates** drop-down list.

Edit Template

- ◆ After creating a view named, Editfor the User model using the Edit template, Visual Studio.NET generates the markup for the view.
- ◆ In this code, ViewData is used to display the values of the Message and CurrentTime key

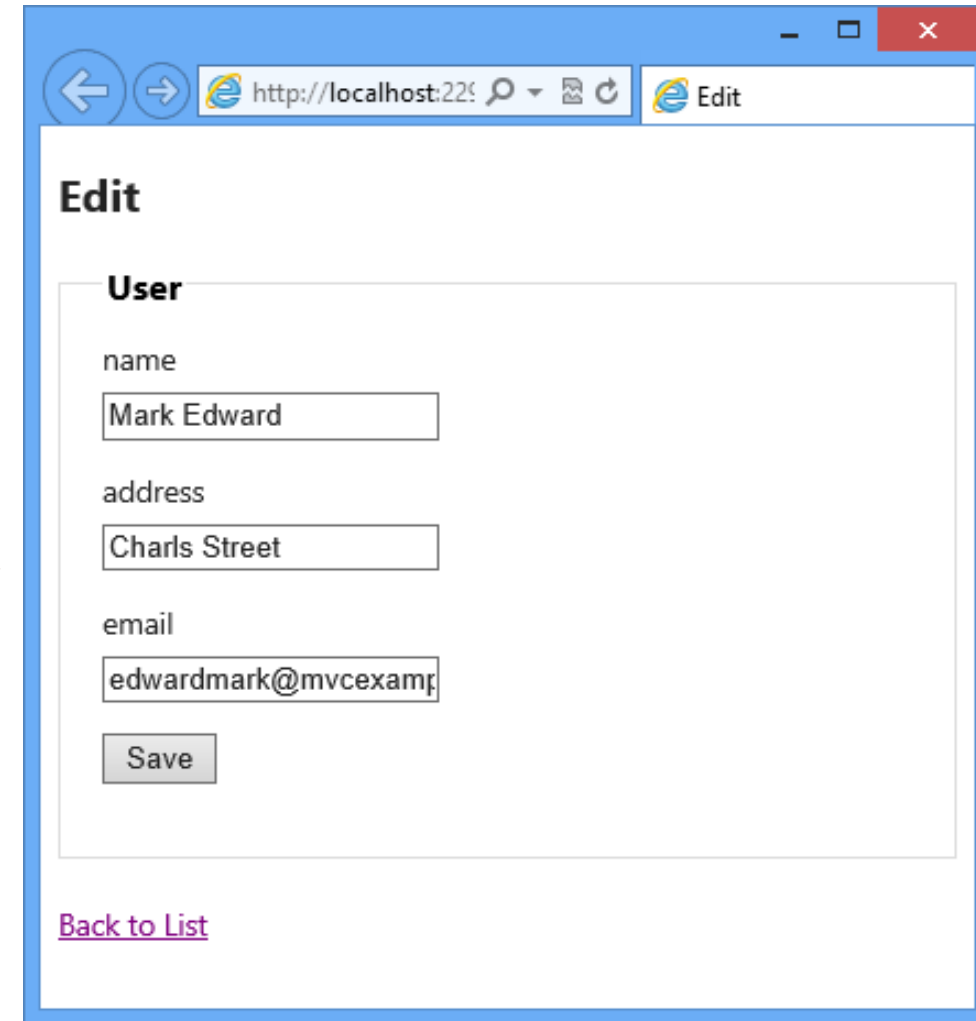
```
@model MVCModelDemo.Models.User
@{ViewBag.Title = "Edit";}
<h2>Edit</h2>
@using (Html.BeginForm()) {
    @Html.ValidationSummary(true)
    <fieldset>
        <legend>User</legend>
        @Html.HiddenFor(model => model.Id)
        <div class="editor-label"> @Html.LabelFor(model => model.name)</div>
        <div class="editor-field">
            @Html.EditorFor(model => model.name)
            @Html.ValidationMessageFor(model => model.name)
        </div>
        <div class="editor-label">@Html.LabelFor(model =>model.address)</div>
```

Edit Template

```
<div class="editor-field">
    @Html.EditorFor(model =>model.address)
    @Html.ValidationMessageFor(model =>model.address)
</div>
<div class="editor-label">@Html.LabelFor(model =>model.email)</div>
<div class="editor-field">
    @Html.EditorFor(model =>model.email)
    @Html.ValidationMessageFor(model =>model.email)
</div>
<p> <input type="submit" value="Save" /></p>
</fieldset>
}
<div>@Html.ActionLink("Back to List", "Index")</div>
@section Scripts {
    @Scripts.Render("~/bundles/jqueryval")
}
```

Edit template

- ◆ The `Html.LabelFor()` helper method displays an HTML label element with the name of the property.
- ◆ The `Html.EditorFor()` helper method displays a textbox to accept the value of a model property.
- ◆ The `Html.ValidationMessageFor()` helper method displays a validation error message.
- ◆ Following figure shows the output of creating the view using the Edit template:



The screenshot shows a web browser window with the address bar displaying `http://localhost:22...` and the page title "Edit". The form itself has a title "Edit" and a section header "User". It contains three text input fields: "name" with the value "Mark Edward", "address" with the value "Charls Street", and "email" with the value "edwardmark@mvcexampl...". Below these fields is a "Save" button. At the bottom of the form, there is a link labeled "Back to List".

Detail Template

- ◆ The Details template allows you to create a view that displays details of the Usermodel.
- ◆ Once you have created the Details()action method in the controller, and a view using the Details template in Visual Studio.NET, it generates the markup for the view.
- ◆ Following code snippet shows the auto-generated markup when you create the view using the Details template:

```
@model MVCModelDemo.Models.User
@{ViewBag.Title = "Details";}
<h2>Details</h2>
<fieldset>
    <legend>User</legend>
    <div class="display-label">@Html.DisplayNameFor(model => model.name)</div>
    <div class="display-field">    @Html.DisplayFor(model => model.name)</div>
```

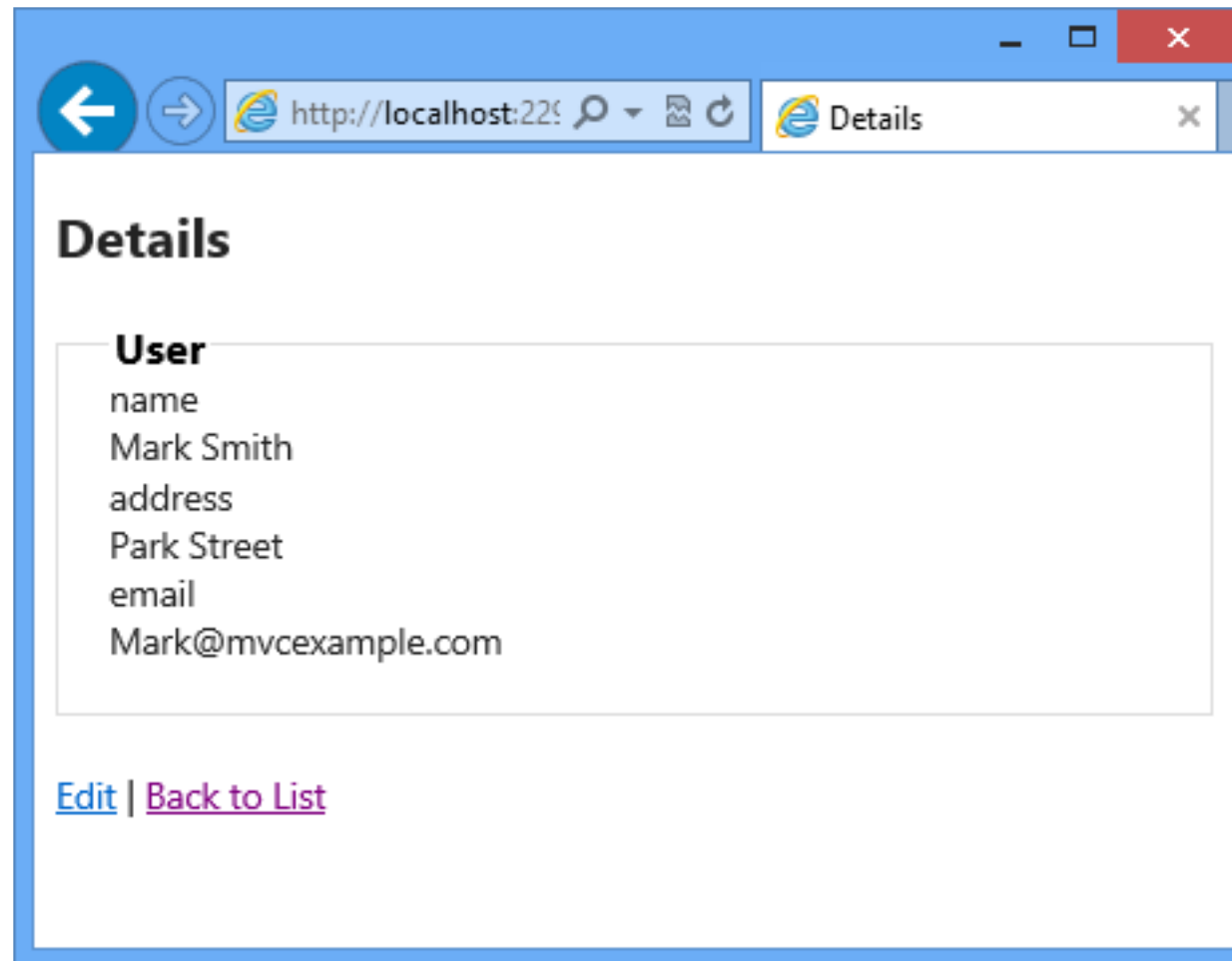
Detail Template

```
<div class="display-label">@Html.DisplayNameFor(model =>model.address)</div>
<div class="display-field">@Html.DisplayFor(model =>model.address)</div>
<div class="display-label">@Html.DisplayNameFor(model =>model.email)</div>
<div class="display-field">@Html.DisplayFor(model =>model.email)</div>
</fieldset>
<p>
    @Html.ActionLink("Edit", "Edit", new { id=Model.Id }) |
    @Html.ActionLink("Back to List", "Index")
</p>
```

- ◆ In this code, the `Html.DisplayNameFor()` helper method displays the name of model properties and the `Html.DisplayFor()` helper method displays the values of the model properties.

Detail Template

- ◆ Following figure shows the output of auto-generated markup for Details template:



Delete Template

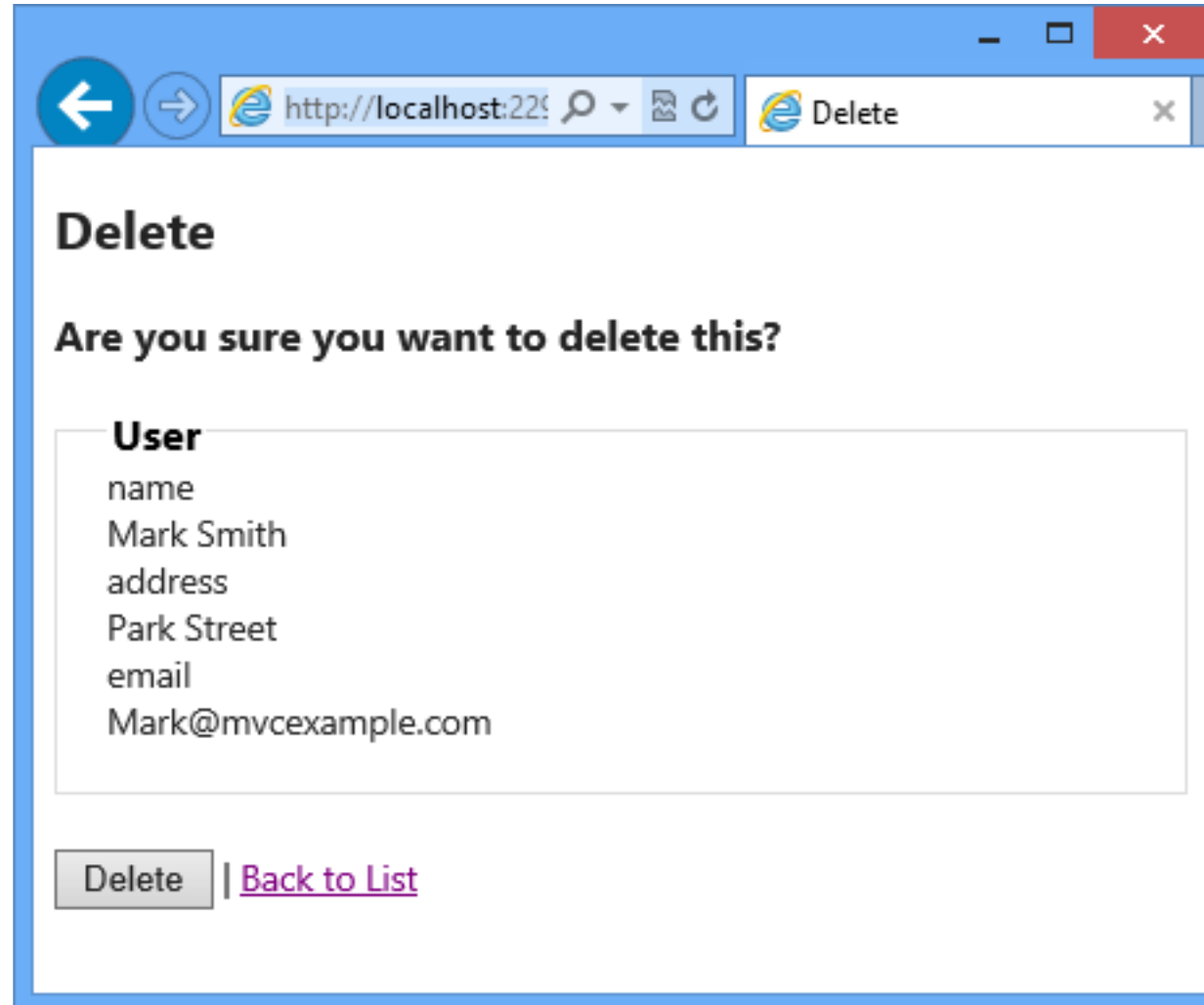
- ◆ Allows generating a view that allows a user to delete an existing object from a data store.
- ◆ To create a view based on the Delete template, first you need to create an action method that passes the model object to be deleted to the view.
- ◆ After you create the Delete()action method, and a view for the User model using the Delete template in Visual Studio.NET, it generates the markup for the view.
- ◆ Following code snippet shows the auto-generated markup when you create a view using the Delete template:

Delete Template

```
@model MVCModelDemo.Models.User
@{ViewBag.Title = "Delete";}
<h2>Delete</h2>
<h3>Are you sure you want to delete this?</h3>
<fieldset>
    <legend>User</legend>
    <div class="display-label">@Html.DisplayNameFor(model => model.name)</div>
    <div class="display-field">@Html.DisplayFor(model => model.name)</div>
    <div class="display-label">@Html.DisplayNameFor(model =>model.address)</div>
    <div class="display-field">@Html.DisplayFor(model =>model.address)</div>
    <div class="display-label">@Html.DisplayNameFor(model =>model.email)</div>
    <div class="display-field">@Html.DisplayFor(model =>model.email)</div>
</fieldset>
@using (Html.BeginForm()) {
    <p>
        <input type="submit" value="Delete" /> | @Html.ActionLink("Back to List", "Index")
    </p>
}
```

Delete Template

- ◆ Following figure shows the output of creating view using the Delete template:



Summary

- ◆ In an ASP.NET MVC application, a model represents data associated with the application. In the MVC pattern, there are three types of models, where each model has specific purpose.
- ◆ The MVC Framework provides helper methods that you can use only in strongly typed views.
- ◆ The process of mapping the data in an HttpRequest object to a model object is known as model binding.
- ◆ The MVC Framework provides a model binder that performs model binding in application.
- ◆ The ASP.NET MVC Framework provides a feature called scaffolding that allows you to generate views automatically.
- ◆ Visual Studio.NET simplifies the process of creating views for an action method using the different scaffolding template.